Highlights

Neighbourhood Structures and Ranking Operators for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez

- Unified extreme critical path framework for neighbourhoods in the interval JSP.
- Feasibility, connectivity and no-loss proofs for any admissible interval ranking.
- Lexicographic interval ranking yields 5–11% narrower makespan intervals on average.
- N_2 and N_8 are the strongest neighbourhoods after irace-tuned tabu search; N_3 worst.
- Irace-tuned TS-N_2 reduces mean relative error $\approx$57% vs best previous IJSP solver.

Neighbourhood Structures and Ranking Operators for the Interval Job Shop Scheduling Problem

Hernán Díaz Rodríguez[a,*]

[a]*Department of Computing, University of Oviedo, Campus de Gijón, Gijón, 33204, Spain*

Abstract

The Job Shop Scheduling Problem with interval-valued processing times (IJSP) models real-world uncertainty where task durations are unknown but bounded. Comparing schedules then requires an interval ranking operator, and the classical notion of critical path must be extended to both extreme scenarios. I formalise *extreme critical paths* — paths critical in the best-case or worst-case scenario graph — and use them to define a neighbourhood $H(\sigma)$ for local search. I prove that $H(\sigma)$ is feasible, connected and loses no improving neighbours under any admissible interval ranking. Five neighbourhood variants (N_1, N_2, N_3, N_{ext}, N_8) are formulated within this framework and combined with four ranking operators (EV, LEX1, LEX2, YX) on 82 benchmark instances of up to 1000 operations. The findings can be summarised as follows: *(i)* after irace-based tabu search tuning, N_2 and N_8 emerge as the two strongest neighbourhoods, with N_{ext}, N_1, and N_3 trailing by progressively larger effects. *(ii)* LEX2 yields makespan intervals 5–11% narrower than the alternatives on average; a float analysis shows LEX2 solutions concentrate critical operations in the worst-case graph, consistent with LEX2 directly tightening the worst-case critical path. *(iii)* On the full 82-instance IJSP benchmark, the irace-tuned TS-N_2 reduces mean RE (%) by 56.6% on the 12 classical instances and 57.7% on the 70 Taillard instances relative to the previous best published solver; the neighbourhood ranking is consistent across all instance size classes.

Keywords: Job shop scheduling, combinatorial optimisation, local search, neighbourhood structures, interval uncertainty, interval ranking operators

*Corresponding author

Email address: diazhernan@uniovi.es (Hernán Díaz Rodríguez)

1. Introduction

The Job Shop Scheduling Problem (JSP) is a flagship combinatorial optimisation problem in operations research, and neighbourhood-based local search has been its most competitive algorithmic paradigm for three decades [1, 2, 3]. The effectiveness of these methods rests almost entirely on neighbourhood design: the critical-block insight of [2] — that reversing interior arcs of a critical block cannot improve the makespan — reduces neighbourhood size by two-thirds without sacrificing solution quality, and the extended neighbourhood of [3] refines this further with a heads-and-tails viability filter. In practice, however, processing times are rarely known precisely at planning time: operator variability, machine condition, and material heterogeneity introduce uncertainty that deterministic models cannot capture.

A natural and tractable representation for such uncertainty is the *closed interval* $[p^-, p^+]$, bounding the minimum and maximum possible durations [4, 5]. The resulting *Interval Job Shop Scheduling Problem (IJSP)* raises two intertwined challenges: comparing schedules requires an *interval ranking operator* [6, 7], since the makespan is now an interval; and the critical-path concept that underpins efficient neighbourhood design must be reconsidered, because different paths may become critical under different realisations of the processing times [8, 9, 10].

1.1. Contributions

This paper makes the following contributions.

1. Formal definition of extreme criticality for IJSP (Section 4). I define a task or arc as *extreme critical* if it lies on a critical path of the best-case graph $G^-(\sigma)$ or the worst-case graph $G^+(\sigma)$, and prove that this definition encompasses all necessarily critical tasks and is contained within the set of possibly critical tasks, making it a computationally tractable and theoretically sound approximation of the general criticality concept of [9].
2. Neighbourhood $H(\sigma)$ based on extreme critical arcs (Section 5). I define $H(\sigma)$ as the set of all feasible processing orders reachable by reversing a single extreme critical disjunctive arc, and prove three structural properties: feasibility (every move yields a feasible order), no loss of improving neighbours (moves outside $H(\sigma)$ cannot improve the makespan

under any admissible ranking), and connectivity (every non-optimal order can reach a global optimum via a finite sequence of moves in $H(\sigma)$).

3. Five neighbourhood variants within the $H(\sigma)$ framework (Section 6). I formulate N_1, N_2, N_3, N_{ext} and a new variant N_8 (boundary arc swaps extended with extra-block reinsertion moves, after [11]) as instances of $H(\sigma)$, characterising their theoretical size and the cost of generating them.
4. Empirical study on 82 benchmark instances (Section 8). Three experimental phases are reported: *(a)* a 5×4 comparison of four ranking operators at a common parameter setting (Section 8.2); *(b)* a head-to-head comparison of the five neighbourhoods after irace-based hyperparameter tuning, isolating the structural effect of the neighbourhood (Section 8.3); and *(c)* a comparison against the state of the art (Section 8.4), showing that the irace-tuned TS-N_2 achieves strictly better average solution quality than the previous best published solver.
5. Structural explanation of LEX2's interval-width effect (Section 8.2). LEX2 consistently yields makespan intervals narrower than the alternatives. A float analysis reveals the structural mechanism: LEX2 concentrates critical operations in the worst-case graph $G^+(\sigma)$, directly tightening the worst-case critical path.

1.2. Paper Organisation

Section 2 provides background on the deterministic JSP and interval ranking operators. Section 3 reviews related work on neighbourhood structures and published IJSP solvers. Section 4 defines extreme criticality. Section 5 defines the neighbourhood $H(\sigma)$ and proves its three structural properties. Section 6 describes the five neighbourhood variants. Section 7 details the tabu search used in Phase B of the experiments. Section 8 reports the experimental study. Section 9 discusses the findings, and Section 10 concludes.

2. Background

2.1. The Deterministic Job Shop Problem

The JSP consists of n jobs $\{J_1, \ldots, J_n\}$ and m machines $\{M_1, \ldots, M_m\}$. Each job J_i is a sequence of operations $O_{i,1}, \ldots, O_{i,m}$, where operation $O_{i,j}$ must be processed on machine M_j for exactly $p_{i,j}$ time units. Operations within a job must respect their precedence order, and each machine processes

at most one operation at a time. The goal is to find a *feasible processing order* σ minimising the makespan $C_{\max}$.

The *solution graph* $G(\sigma) = (V, A \cup D(\sigma))$ represents a feasible order: V is the set of operations augmented with a dummy source node S and sink node T; A contains precedence arcs within jobs; and $D(\sigma)$ contains disjunctive arcs imposed by the order σ on each machine. The makespan equals the length of the longest path in $G(\sigma)$. The classical insight of [1] is that the neighbourhood for local search should restrict to reversals of arcs on critical paths, ensuring feasibility, while [2] shows that reversals of *interior* arcs of a critical block cannot improve the makespan, allowing further pruning.

2.2. Interval Arithmetic and Ranking Operators

When processing times are uncertain, I represent each as $\mathbf{p} = [p^-, p^+] \in \mathbb{IR}$, where $\mathbb{IR}$ denotes the set of closed intervals. The IJSP requires three arithmetic operations on $\mathbb{IR}$ [4]:

$$\mathbf{a} + \mathbf{b} = [a^- + b^-,\, a^+ + b^+], \tag{1}$$

$$\max(\mathbf{a}, \mathbf{b}) = [\max(a^-, b^-),\, \max(a^+, b^+)]. \tag{2}$$

Since there is no natural total order on $\mathbb{IR}$, comparing schedules requires choosing a *ranking operator*. For any interval $\mathbf{a} = [a^-, a^+]$, define its *midpoint* $m(\mathbf{a}) = \frac{a^- + a^+}{2}$ and *width* $w(\mathbf{a}) = a^+ - a^-$. I consider four rankings widely used in the literature [6, 12, 5]:

$$\mathbf{a} \leq_{\text{LEX1}} \mathbf{b} \iff a^- < b^- \vee (a^- = b^- \wedge a^+ \leq b^+) \tag{3}$$

$$\mathbf{a} \leq_{\text{LEX2}} \mathbf{b} \iff a^+ < b^+ \vee (a^+ = b^+ \wedge a^- \leq b^-) \tag{4}$$

$$\mathbf{a} \leq_{\text{YX}} \mathbf{b} \iff m(\mathbf{a}) < m(\mathbf{b}) \vee \big(m(\mathbf{a}) = m(\mathbf{b}) \wedge w(\mathbf{a}) \leq w(\mathbf{b})\big) \tag{5}$$

$$\mathbf{a} \leq_{\text{EV}} \mathbf{b} \iff m(\mathbf{a}) \leq m(\mathbf{b}) \tag{6}$$

LEX1, LEX2 and YX are *admissible* total orders on $\mathbb{IR}$ [6]: they refine the partial product order $\leq_2$ defined by $\mathbf{a} \leq_2 \mathbf{b} \iff a^- \leq b^- \wedge a^+ \leq b^+$. EV ranks intervals by their midpoint $m(\mathbf{a})$ and induces a total preorder rather than a strict total order, since two distinct intervals may share the same midpoint; it also refines $\leq_2$. This refinement property is essential for the theoretical guarantees of Section 5: any move that simultaneously decreases both $C^-_{\max}$ and $C^+_{\max}$ is an improvement under all four rankings considered here.

3. Related Work

Neighbourhood structures for fuzzy and interval JSP.. The first neighbourhood for the fuzzy/interval JSP was proposed in [10], extending the van Laarhoven neighbourhood [1] to use possibly critical arcs rather than necessarily critical ones. This corresponds to what I call N_1 in the present framework. Subsequent work introduced N_2 as a filtered version of N_1 that retains only block-boundary arcs [13], reducing neighbourhood size by $\sim$64% with statistically identical solution quality on the fuzzy JSP. N_3 extends N_2 with three-task rearrangements at block boundaries [14]. For the related fuzzy job shop, evolutionary tabu search combining these neighbourhood structures with genetic operators has been proposed for due-date satisfaction objectives [15]. Lexicographic goal-programming strategies have also been explored for the green fuzzy FJSP with vague production targets [16] and for the green interval FJSP combining makespan and energy objectives [17]. Prior approaches to the IJSP specifically have combined neighbourhood search with genetic algorithms for makespan minimisation [18] and tardiness [19], providing early evidence that the choice of ranking operator influences solution quality.

The Nowicki–Smutnicki extended neighbourhood [3], adapted here as N_{ext}, was originally proposed for the crisp JSP and achieves strong empirical performance by additionally considering interior block arcs that pass a heads-and-tails viability check. Its adaptation to the IJSP is a contribution of this work, exploiting the fact that the boundary-arc result of [2] can be lifted simultaneously to both extreme graphs $G^{-}(\sigma)$ and $G^{+}(\sigma)$. A fifth variant, N_8, is also proposed here: it extends N_2 with extra-block reinsertion moves that relocate critical block endpoints to positions outside the block, adapting the idea of [11] — originally developed for the crisp JSP — to the interval setting. For the deterministic JSP and FJSP, general frameworks for feasibility checking and quality evaluation of neighbourhood moves have been proposed in [20, 21]; analogous efficiency improvements for the interval setting remain an open direction. In a complementary line, machine learning methods have been proposed to estimate quantile-based risk measures of the IJSP makespan [22].

Metaheuristic solvers for the IJSP.. The fast-elitist Artificial Bee Colony (*fEABC*) [23] established best-known results on the 82-instance benchmark used here, combining ABC exploration with hill-climbing local search. The

Elitist-Selection ABC (*ESABC*) [24] further improved solution quality by incorporating a simulated-annealing diversity mechanism, at the cost of increased runtime. These two methods, together with the earlier GA of [18], constitute the state of the art against which the proposed tabu search is compared in Section 8.4.

4. Extreme Criticality for IJSP

4.1. Configurations and Extreme Scenario Graphs

A *configuration* $\omega = (p_1, \ldots, p_o)$, where $o = n \times m$, assigns a deterministic duration $p_i \in \mathbf{p}_i$ to each operation i; let Ω denote the set of all configurations. Given a feasible processing order σ, two *extreme scenario graphs* are of particular interest:

- $G^-(\sigma)$: identical to $G(\sigma)$ but with each arc (x, y) weighted by p_x^- (best case).
- $G^+(\sigma)$: identical to $G(\sigma)$ but with each arc (x, y) weighted by p_x^+ (worst case).

The makespan interval satisfies

$$\mathbf{C}_{\max}(\sigma) = [C^-_{\max}(\sigma),\, C^+_{\max}(\sigma)], \tag{7}$$

where $C^-_{\max}(\sigma)$ is the length of the longest path in $G^-(\sigma)$ and $C^+_{\max}(\sigma)$ is the length of the longest path in $G^+(\sigma)$. An illustrative example is shown in Figure 1: two jobs (J_1: $O_{11}(M_1, [2, 6]) \to O_{12}(M_2, [2, 3])$; J_2: $O_{21}(M_2, [4, 4]) \to O_{22}(M_1, [1, 4])$), with schedule σ fixing M_1 order (O_{11}, O_{22}) and M_2 order (O_{21}, O_{12}). In $G^-(\sigma)$ the critical path runs through the M_2 disjunctive arc ($C^-_{\max} = 6$); in $G^+(\sigma)$ it runs through the M_1 disjunctive arc ($C^+_{\max} = 10$). The critical arcs in the two extreme graphs differ, motivating the need to consider both.

4.2. Heads, Tails and Floats

For each extreme graph $G^*(\sigma)$ ($* \in \{-, +\}$), the *head* r^*_x of task x is the length of the longest path from the dummy start node to x, and the *tail* q^*_x is the length of the longest path from the completion of x to the end node. The *float* of x in $G^*(\sigma)$ is

$$F^*_x(\sigma) = C^*_{\max}(\sigma) - r^*_x - p^*_x - q^*_x. \tag{8}$$

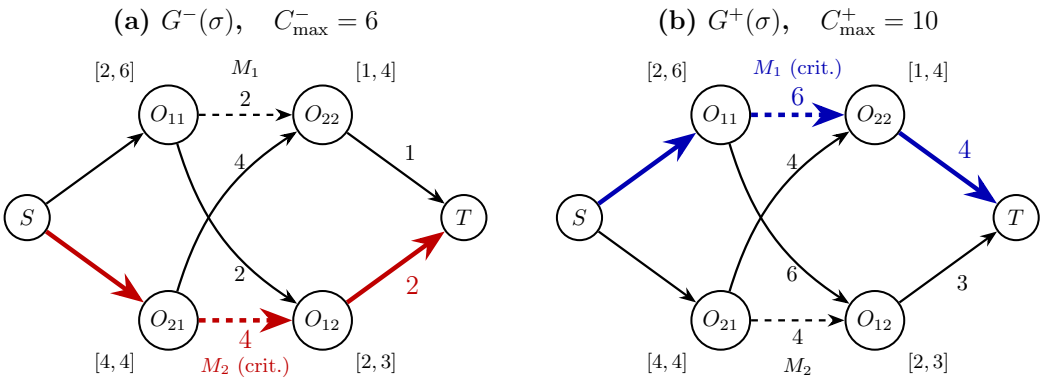


Figure 1: Illustrative example. (a) Best-case graph $G^-(\sigma)$ with its critical path highlighted in red. (b) Worst-case graph $G^+(\sigma)$ with its (different) critical path highlighted in blue. Precedence arcs are drawn solid, disjunctive arcs dashed.

A task is critical in $G^*(\sigma)$ iff $F^*_x(\sigma) = 0$. The general problem of computing the minimum float $\min_{\omega\in\Omega} F_x(\sigma, \omega)$ over all configurations — which equals zero iff x is possibly critical — is NP-hard [25, 26]; efficient branch-and-bound algorithms for this harder problem are given in [27]. The extreme-scenario floats F^-_x and F^+_x used here are computable in linear time and provide the tractable approximation exploited in Sections 5 and 6.

4.3. Definition and Connection to Possibly/Necessarily Critical Tasks

Definition 1 (Extreme critical path)**.** *A path P in $G(\sigma)$ is* extreme critical *if it is critical in $G^-(\sigma)$ or in $G^+(\sigma)$. A node, arc or block is extreme critical if it lies on an extreme critical path.*

This definition connects to the general criticality concepts of [25, 9] as follows. A task x is *possibly critical* under σ if there exists $\omega \in \Omega$ such that $F_x(\sigma, \omega) = 0$; it is *necessarily critical* if $F_x(\sigma, \omega) = 0$ for every $\omega \in \Omega$ [25, 9].

Proposition 1. *Every extreme critical task is possibly critical. Every necessarily critical task is extreme critical.*

Proof. (i) Extreme critical ⇒ possibly critical. Let task x be extreme critical; then $F^-_x(\sigma) = 0$ or $F^+_x(\sigma) = 0$. The extreme configurations $\omega^- = (p^-_i)_i$ and $\omega^+ = (p^+_i)_i$ both belong to Ω (since $p^\pm_i \in [p^-_i, p^+_i]$ by definition). In case $F^-_x(\sigma) = 0$: the float of x under configuration ω^- equals $F_x(\sigma, \omega^-) =$

$C^-_{\max}(\sigma) - r^-_x - p^-_x - q^-_x = F^-_x(\sigma) = 0$, so x lies on a critical path for $\omega^- \in \Omega$, making it possibly critical [25, 9]. The case $F^+_x(\sigma) = 0$ is analogous with ω^+.

(ii) Necessarily critical ⇒ extreme critical. If x is necessarily critical, then $F_x(\sigma, \omega) = 0$ for every $\omega \in \Omega$. Taking $\omega = \omega^- \in \Omega$ gives $F^-_x(\sigma) = 0$, so x is critical in $G^-(\sigma)$, hence extreme critical. □

The practical consequence is that extreme criticality offers a computationally tractable approximation to the exact criticality predicates: computing critical paths in $G^-(\sigma)$ and $G^+(\sigma)$ each costs $O(|A \cup D(\sigma)|)$ via standard longest-path algorithms, while computing the full set of possibly critical tasks is NP-hard in general [25, 9].

5. The Neighbourhood $H(\sigma)$ and Its Properties

5.1. Definition

Definition 2 (Neighbourhood $H(\sigma)$)**.** *Let $\sigma_{(v)}$ denote the processing order obtained from σ by reversing a single disjunctive arc $v \in D(\sigma)$. The neighbourhood is*

$$H(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is extreme critical}\}. \tag{9}$$

For comparison, the corresponding neighbourhood based on possibly *critical arcs is*

$$H'(\sigma) = \{\sigma_{(v)} : v \in D(\sigma) \text{ is possibly critical}\}. \tag{10}$$

By Proposition 1, $H(\sigma) \subseteq H'(\sigma)$, with strict inclusion in general.

5.2. No Loss of Improving Neighbours

Proposition 2. *Let $\sigma \in \Sigma$ be a feasible order and let $v \in D(\sigma)$ be a disjunctive arc that is* not *extreme critical in $G(\sigma)$. Then*

$$C^-_{\max}(\sigma) \le C^-_{\max}(\sigma_{(v)}) \quad \text{and} \quad C^+_{\max}(\sigma) \le C^+_{\max}(\sigma_{(v)}), \tag{11}$$

and consequently $\mathbf{C}_{\max}(\sigma) \le_R \mathbf{C}_{\max}(\sigma_{(v)})$ for every admissible ranking R.

Proof. Fix $* \in \{-, +\}$. Since (x, y) is not extreme critical, no critical path of $G^*(\sigma)$ contains it; any such path, of length $C^*_{\max}(\sigma)$, remains a valid path in $G^*(\sigma_{(v)})$ (whose only arc change is $(x, y) \mapsto (y, x)$), hence $C^*_{\max}(\sigma_{(v)}) \ge C^*_{\max}(\sigma)$. Applying this to both extreme graphs gives $\mathbf{C}_{\max}(\sigma) \le_2 \mathbf{C}_{\max}(\sigma_{(v)})$, and since every admissible R refines $\le_2$, the claim follows. □

Corollary 1. *Neighbours in $H'(\sigma) \setminus H(\sigma)$ can never improve the makespan under any of the four rankings considered.*

Corollary 1 justifies using $H(\sigma)$ instead of the larger $H'(\sigma)$: the reduction is gained at zero cost in solution quality.

5.3. Feasibility

Theorem 1 (Feasibility)**.** *For every feasible $\sigma \in \Sigma$, the reversal of any extreme critical arc $v = (x, y) \in D(\sigma)$ produces a feasible processing order $\sigma_{(v)} \in \Sigma$. Hence $H(\sigma) \subseteq \Sigma$.*

Proof. Suppose $G(\sigma_{(v)})$ contains a directed cycle. Since $G(\sigma)$ is acyclic and only (x, y) has been removed and (y, x) added, the cycle traverses (y, x), yielding a directed path π from x to y in $G(\sigma)$ not using (x, y). In standard JSP, x and y lie on the same machine but in different jobs, so no precedence arc joins them; π has at least one intermediate operation u, and its length in $G^*(\sigma)$ satisfies

$$\ell(\pi) \;=\; p^*_x + p^*_u + \cdots \;>\; p^*_x.$$

But (x, y) lies on a critical path of $G^*(\sigma)$, so the head of y is tight: $r^*_y = r^*_x + p^*_x$. The path π then gives $r^*_y \geq r^*_x + \ell(\pi) > r^*_x + p^*_x = r^*_y$, a contradiction. □

5.4. Connectivity

Theorem 2 (Connectivity)**.** *For every non-optimal $\sigma \in \Sigma$, there exists a finite sequence of moves in $H(\sigma)$ leading to a globally optimal order $\sigma^\star$.*

Proof. Fix any globally optimal $\sigma^* \in \Sigma^*$ and define the *inversion count*

$$M(\sigma, \sigma^*) \;=\; \big|\{(x, y) \in D(\sigma) : (y, x) \in D(\sigma^*)\}\big|,$$

the number of machine-order disagreements; $M = 0$ iff $\sigma = \sigma^*$.

Key lemma. If λ is non-optimal, there exists an extreme critical $(x, y) \in D(\lambda)$ with $(y, x) \in D(\sigma^*)$.

Proof. Non-optimality implies $\mathbf{C}_{\max}(\lambda) \not\leq_2 \mathbf{C}_{\max}(\sigma^*)$ (else $\leq_R$ follows by refinement), so WLOG $C^-_{\max}(\lambda) > C^-_{\max}(\sigma^*)$. Let P be a critical path in $G^-(\lambda)$. If every disjunctive arc of P lay in $D(\sigma^*)$, P would be a valid path in $G^-(\sigma^*)$ of length $> C^-_{\max}(\sigma^*)$, a contradiction. So P contains some $(x, y) \in D(\lambda) \setminus D(\sigma^*)$; since machine orientations are mutually exclusive, $(y, x) \in D(\sigma^*)$, and (x, y) is extreme critical by membership in P. □lemma

Construction. Build $\lambda_0 = \sigma$; while λ_k is non-optimal, apply the key lemma to obtain (x, y) and set $\lambda_{k+1} = (\lambda_k)_{(x,y)} \in H(\lambda_k)$ (feasible by Theorem 1). Each reversal removes (x, y) from $D(\lambda_k)$ (one fewer inversion) and adds (y, x) (already in $D(\sigma^*)$, no new inversion), so $M(\lambda_{k+1}, \sigma^*) = M(\lambda_k, \sigma^*) - 1$. Since $M \geq 0$, the loop terminates in at most $M(\sigma, \sigma^*)$ steps at a globally optimal λ_k. □

6. Neighbourhood Variants for IJSP

I instantiate five concrete neighbourhood variants within the $H(\sigma)$ framework, differing in which subset of extreme critical arcs each one considers. Within a critical block $B = (o_1, \ldots, o_k)$, an arc (o_i, o_{i+1}) is a *boundary arc* if $i = 1$ or $i = k - 1$, and an *interior arc* otherwise. Figure 2 illustrates a critical block and the arcs included by each variant.

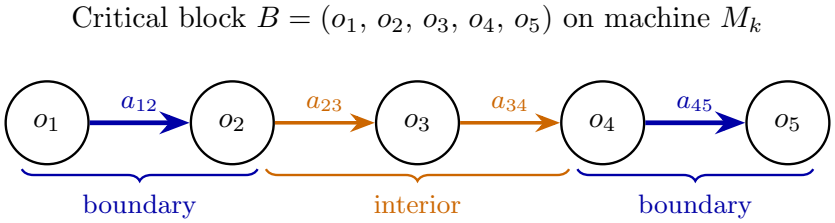
All five variants inherit the three structural guarantees of Section 5 (feasibility, no-loss of improving neighbours, and connectivity) for their arc-swap component, either directly (when only interior arcs are excluded, by Proposition 2) or by superset relation to a variant that does. N_8 additionally introduces reinsertion moves that fall outside the framework; their feasibility is verified at runtime, and N_8 is therefore included as an exploratory variant alongside the four arc-swap variants.

6.1. N_1: All Extreme Critical Arcs

N_1 includes every disjunctive arc (x, y) such that x immediately precedes y on the same machine in $G(\sigma)$ *and* the arc lies on a critical path of $G^-(\sigma)$ or $G^+(\sigma)$. No positional filter is applied within the block: both boundary and interior arcs of every extreme critical block are candidates. Generation cost: $O(|E|)$ per extreme graph.

6.2. N_2: Block-Boundary Filter

N_2 restricts N_1 to arcs at the *boundary* of each extreme critical block. A block $B = \{o_1, \ldots, o_k\}$ is a maximal contiguous subsequence of operations on the same machine that all lie on a single extreme critical path. N_2 retains the two boundary arcs (o_1, o_2) and (o_{k-1}, o_k) and discards interior arcs (o_i, o_{i+1}), $2 \leq i \leq k - 2$. This is the direct adaptation of [2] to the interval setting: by Proposition 2 applied simultaneously to $G^-(\sigma)$ and $G^+(\sigma)$, the discarded interior arcs cannot carry improving moves under any of the four rankings, so N_2 inherits the same theoretical guarantees as N_1.



	a_{12}	a_{23}	a_{34}	a_{45}
N_1	•	•	•	•
N_2	•	–	–	•
N_3	•	•	•	• †
N_{ext}	•	$?_v$	$?_v$	•
N_8	•	–	–	• ‡

• always included; – always excluded; $?_v$ conditional on `isViableSwap` (N_{ext}).
† N_3 and ‡ N_8 additionally generate moves not shown here (Sections 6.3, 6.5).

Figure 2: Critical block $B = (o_1, \ldots, o_5)$ on machine M_k and arcs considered by each neighbourhood variant.

6.3. N_3: Subsequence Reversals

N_3 augments the N_2 boundary arc swaps with three rearrangement types applied to *boundary triples* of consecutive extreme-critical tasks. For each triple (x, y, z) of successive operations on the same machine that are all extreme critical and span a block boundary (i.e. x is the first task in its block or z is the last), N_3 generates three candidate moves: a cyclic left-shift $(x, y, z \to y, z, x)$, a cyclic right-shift $(x, y, z \to z, x, y)$, and a full reversal $(x, y, z \to z, y, x)$. Each candidate is checked for feasibility before being retained. The resulting neighbourhood strictly contains N_2.

6.4. N_{ext}: Extended Nowicki–Smutnicki Neighbourhood

N_{ext} adapts the extended neighbourhood of [3]. It starts from N_2 (boundary arcs unconditionally) and additionally considers interior block arcs that pass a heads-and-tails viability check, denoted `isViableSwap`:

- Boundary arcs (o_1, o_2) and (o_{k-1}, o_k) are always included (as in N_2).
- Interior arcs (o_i, o_{i+1}) with $2 \leq i \leq k-2$ are included only if the `isViableSwap` check returns true.

The check `isViableSwap`(x, y) computes an interval lower bound $\mathbf{LB}(x, y)$ on the post-swap makespan $\mathbf{C}_{\max}(\sigma_{(x,y)})$ by recalculating only the local heads and tails of x and y on both extreme graphs and combining the two per-operation estimates ($\mathbf{r}_x + \mathbf{p}_x + \mathbf{q}_x$ and $\mathbf{r}_y + \mathbf{p}_y + \mathbf{q}_y$, all intervals) componentwise with interval max. The swap is discarded if $\mathbf{LB}(x, y)$ is not better than the current incumbent under the active ranking. Because $\mathbf{LB}(x, y) \leq_2 \mathbf{C}_{\max}(\sigma_{(x,y)})$ and every admissible ranking refines $\leq_2$, discarding is theoretically safe.

6.5. N_8: Block-Boundary Swaps with Extra-Block Reinsertion

N_8 is an exploratory variant that extends N_2 with *extra-block reinsertion* moves, adapting the neighbourhood of [11] to the interval setting. For a critical block $B = (o_1, \ldots, o_k)$ on a machine, with machine predecessor o_0 (if any) and machine successor o_{k+1} (if any), N_8 includes all N_2 boundary arc swaps and additionally considers four reinsertion candidates per block:

- move o_1 to just after o_k (between o_k and o_{k+1});
- move o_k to just before o_1 (between o_0 and o_1);
- move o_1 to the head of the machine (before all tasks on that machine);
- move o_k to the tail of the machine (after all tasks on that machine).

Each reinsertion candidate is first filtered by a heads-and-tails estimate $\mathbf{LB}_8$ of the post-reinsertion makespan, computed from cached heads and tails of the moved operation and its old and new machine neighbours (no graph propagation). The candidate is admitted unless $\mathbf{LB}_8 \geq_2 \mathbf{C}_{\max}^{\text{best}}$, i.e., dominated in both bounds. Admitted candidates are then evaluated exactly by applying the move and propagating head changes via breadth-first propagation through machine and job successors, with early termination if the running bound becomes worse than the incumbent under the active ranking.

The motivation is the empirical question of whether extra-block moves benefit interval scheduling. As shown in Section 8.3, the answer is search-strategy dependent: N_8 underperforms when paired with hill climbing but matches N_2 when paired with irace-tuned tabu search; Section 9.4 explains why.

7. Tabu Search with the $H(\sigma)$ Neighbourhoods

The experimental study of Section 8 uses fEABC [23]—a hybrid ABC/PSO metaheuristic with an embedded local search applied each generation to a fraction of the population—as the base solver, with solutions encoded as job-order permutations with repetition and decoded via an insertion-based Schedule Generation Scheme (SGS). The embedded local-search component is either a first-improvement hill climbing or the short-term-memory tabu search (TS) of Algorithm 1; the choice between them is fixed in Section 8.

Algorithm 1 Tabu search with the $H(\sigma)$ neighbourhood (LS_Tabu).

Require: σ_0, N, R; caps $L_{\max}, B_{\max}, T_{\max}$
Ensure: best solution σ^*

$\sigma \leftarrow \sigma_0$; $\sigma^* \leftarrow \sigma_0$; $L \leftarrow \emptyset$; $b \leftarrow 0$; $m_{\text{last}} \leftarrow$ null
while $b < B_{\max}$ **and** runtime $< T_{\max}$ **do**
  $C \leftarrow N(\sigma)$; sort C ascending by heads-and-tails LB $\widehat{\text{LB}}$
  $m^\star \leftarrow$ null; $v^\star \leftarrow +\infty$
  for all $c \in C$ in sorted order **do**
    if $\widehat{\text{LB}}(c) \geq_R v^\star$ **then break** ▷ LB-filter
    end if
    $v \leftarrow$ exact value of c under R
    tabu $\leftarrow$ (reverse of c) $\in L$; undo $\leftarrow c \equiv \text{reverse}(m_{\text{last}})$
    if $v <_R v^\star$ **and** ($\neg$tabu **or** $v <_R \text{fit}(\sigma^*)$) **and** $\neg$undo **then**
      $m^\star \leftarrow c$; $v^\star \leftarrow v$ ▷ aspiration
    end if
  end for
  if $m^\star =$ null **then break** ▷ dead end
  end if
  apply $m^\star$ to σ; L.push_front($m^\star$); $m_{\text{last}} \leftarrow m^\star$; **if** $|L| > L_{\max}$ **then** L.pop_back()
  if $\text{fit}(\sigma) <_R \text{fit}(\sigma^*)$ **then**
    $\sigma^* \leftarrow \sigma$; $b \leftarrow 0$
  else
    $b \leftarrow b + 1$
  end if
end while
return σ^*

Each TS call is invoked by fEABC on one of its population individuals σ_0. The algorithm maintains two solutions: the one currently being explored (σ) and the best found so far in this call (σ^*). Both are initialised to σ_0 (line 1). The tabu list L starts empty. At every outer iteration the candidate set $C = N(\sigma)$ is regenerated from the current solution and sorted by a heads-and-tails interval lower bound $\widehat{\mathrm{LB}}$ (the same bound used by N_{ext}'s viability filter, Section 6.4); the inner candidate scan then tracks the best surviving candidate in $v^\star$ (line 4). The early-termination test on line 6 exploits the sorted order: once $\widehat{\mathrm{LB}}(c)$ would no longer improve on $v^\star$, the remainder of the sorted list is dominated and can be skipped. The acceptance test on line 10 combines best-improvement with two restrictions—a candidate whose reverse is in L is *tabu*, and the reverse of the last accepted move is blocked to prevent immediate undo—and admits a tabu candidate only when it strictly improves σ^*, the standard *aspiration* criterion. The tabu list is a FIFO queue with no fixed maximum size ($L_{\max}$), so each accepted move stays tabu for the rest of the local-search call (bounded by $B_{\max}$ or $T_{\max}$). The wall-clock cap $T_{\max}$ is fixed at 2 s per call, while the bad-iteration cap $B_{\max}$ is the per-neighbourhood irace-tuned *Bad-iter* value of Table 1. The improved solution returned at line 23 is reinserted into the fEABC population by Lamarckian replacement.

8. Experimental Study

The experimental study is organised in three phases that address distinct questions. Phase A (Section 8.2) compares the four ranking operators at a *common* hyperparameter setting, focusing on expected makespan (quality) and interval width (uncertainty). Phase B (Section 8.3) then fixes the ranking operator and gives each of the five neighbourhoods its own irace-tuned configuration, comparing the neighbourhoods on equal footing. Finally, Phase C (Section 8.4) compares the best Phase B configuration against the state of the art in IJSP, providing external validation of the proposed approach.

8.1. Setup Common to All Phases

Instances.. The benchmark suite contains 82 instances split into two groups. The first group (*classical benchmarks*) comprises 12 instances selected from the OR-Library [28]: abz7, abz8, abz9 (20×15); ft10 (10×10), ft20 (20×5); la21, la24, la25 (15×10), la27, la29 (20×10), la38, la40 (15×15). The second group comprises 70 instances from the Taillard benchmark [29], covering

seven size classes with 10 instances each: 15×15 (225 operations), 20×15 (300), 20×20 (400), 30×15 (450), 30×20 (600), 50×15 (750) and 50×20 (1 000). All 82 instances are derived from their deterministic counterparts by perturbing each nominal processing time p^*_{ij} with a $\pm 7.5\%$ uniform symmetric interval, $\mathbf{p}_{ij} = [0.925\, p^*_{ij},\ 1.075\, p^*_{ij}]$ (15% relative width).

Hardware and software.. All experiments were executed on an Intel Xeon E5-2680 v4 workstation (14 cores / 28 threads, 2.40 GHz, 16 GB RAM) running Windows 10 with a WSL2 Linux environment, dispatching up to 24 processes in parallel (one per thread). The fEABC algorithm was implemented in C++14 and compiled with GCC; statistical analysis was performed in R.

Hyperparameter configuration.. Phase A adopts the configuration validated in the fEABC paper [23]: JOX (Job Order Crossover), inversion mutation, the food-source abandonment limit NTmax = 20 (a candidate solution not improved after NTmax trials is discarded and reinitialised), a population of 250 with elite size 40, and a first-improvement hill-climbing local search applied to 75% of individuals per generation. The search terminates after 25 consecutive generations without improvement in the global best.

Phase B replaces the hill-climbing local search by the tabu search of Section 7 and tunes six parameters independently per neighbourhood using irace [30] (R 4.5.3): crossover operator, mutation operator, elite size, NTmax, local-search target fraction, and the bad-iteration limit $B_{\max}$ (the per-call cap on consecutive non-improving tabu iterations of Algorithm 1). The tuning used a budget of 1 500 candidate evaluations per neighbourhood with a Friedman-test racing criterion. To avoid overfitting to any particular size class, irace was run on a stratified random subset of 20 training instances ($\approx 24\%$ of the benchmark): 3 of the 12 classical instances and 2–3 of the 10 Taillard instances per size class were drawn uniformly at random within each stratum (full list in the supplementary material [31]). All 82 instances—including the 62 held out from tuning—are then evaluated in the unbiased comparison reported below. The remaining parameters are fixed: population 250, tabu-list minimum size 3, and LEX2 ranking (selected in Phase A; Section 8.2). Table 1 reports the resulting configurations. Phase B inherits the Phase A termination rule and additionally imposes a wall-clock limit of 900 s (15 minutes) per run; the full evaluation comprised 12 300 runs (five neighbourhoods $\times$ 82 instances $\times$ 30 independent replications, the latter being the standard for every (instance, configuration) pair throughout

the study), and required together with the tuning approximately nine days of wall-clock time.

Table 1: Irace-tuned hyperparameter configurations for Phase B. Parameters not listed are fixed: population 250, tabu-min-size 3, LEX2 ranking. *LS target*: fraction of individuals receiving local search per generation. *Bad-iter*: maximum consecutive non-improving tabu iterations before the local search terminates.

	Crossover	Mutation	Elite	NTmax	LS target	Bad-iter
N_1	JOX	insertion	33	11	85.3%	16
N_2	JOX	insertion	50	24	100%	20
N_3	GOX	swap	48	25	82.8%	30
N_{ext}	JOX	swap	39	17	76.2%	23
N_8	JOX	swap	39	27	100%	21

Statistics.. Comparisons use the Friedman test (non-parametric repeated-measures ANOVA; the statistic follows a χ^2 distribution with $k-1$ degrees of freedom for k groups) followed by pairwise Wilcoxon signed-rank tests with Holm–Bonferroni correction. Effect size is reported as $r = |Z|/\sqrt{N}$, where Z is the standardised Wilcoxon test statistic and N is the number of paired observations, with the conventional thresholds (small ≥ 0.1, medium ≥ 0.3, large ≥ 0.5). Each (instance, run) pair forms a paired block. The metric is the midpoint of the makespan interval, $(C^-_{\max} + C^+_{\max})/2$.

8.2. Phase A: Operator Calibration

Phase A is preparatory rather than central: its purpose is to identify the ranking operator that minimises makespan interval uncertainty at negligible midpoint cost, so that it can be fixed for the neighbourhood comparison in Phase B. The 5×4 design (five neighbourhoods, four operators, common first-improvement hill-climbing local search) gives 49,200 runs and also establishes a neutral baseline.

Hardware calibration.. To calibrate this cluster against the hardware used in the published IJSP solvers compared in Section 8.4, I ran the fEABC+HC configuration used here—identical in algorithm and parameters to the one reported in [23], which used an Intel Xeon Gold 6132 at 2.60 GHz under native Linux—on the same 12 classical instances and compared average runtimes: the ratio is approximately 2× (geometric mean 2.10, range 1.4–2.8), indicating this cluster runs the identical workload roughly twice as slow. Runtime

Table 2: Phase A: mean midpoint (mid.) and median wall-clock time t (s) per run, by neighbourhood and ranking operator (82 instances × 30 runs). For midpoint, lower is better.

	EV		LEX1		LEX2		YX	
Neighbourhood	mid.	t	mid.	t	mid.	t	mid.	t
N_1	1888.9	52	1894.6	33	1889.5	37	1887.0	32
N_2	1888.7	29	1895.2	41	1889.2	33	1887.1	29
N_3	1907.3	194	1914.2	206	1908.5	254	1904.9	201
N_8	1890.5	30	1894.6	33	1891.3	36	1887.5	31
N_{ext}	1889.0	33	1891.0	85	1895.5	65	1887.2	33

Table 3: Phase A: pairwise Wilcoxon tests for ranking operators ($n = 2460$ paired blocks, Holm–Bonferroni corrected).

A	B	Mean A	Mean B	Diff	p_{adj}	r	Magnitude
LEX1	YX	1909.5	1901.5	+8.0	$< 10^{-3}$	0.509	large
EV	LEX1	1903.3	1909.5	−6.2	$< 10^{-3}$	0.415	medium
LEX1	LEX2	1909.5	1903.9	+5.6	$< 10^{-3}$	0.349	medium
LEX2	YX	1903.9	1901.5	+2.4	$< 10^{-3}$	0.159	small
EV	YX	1903.3	1901.5	+1.8	$< 10^{-3}$	0.131	small
EV	LEX2	1903.3	1903.9	−0.6	0.197	0.026	negligible

figures in Section 8.4 should be interpreted accordingly; RE (%) comparisons are hardware-independent.

Midpoint quality.. I begin by comparing the four operators on solution quality, taken as the makespan-interval midpoint (lower is better). The Friedman test across operators yields $\chi^2(3) = 694.96$, $p < 10^{-150}$, confirming that the operators are not all equivalent. Table 2 reports the global mean midpoint and median runtime; pairwise tests (Table 3) show that LEX1 is significantly worse than the other three (medium-to-large effects), EV and LEX2 are statistically indistinguishable ($p = 0.197$, $r = 0.026$), and YX leads on average but its advantage over EV and LEX2 carries only small effects ($r \leq 0.131$). The choice of operator has no meaningful effect on wall-clock time: Kruskal–Wallis tests return $p > 0.48$ for four of the five neighbourhoods, and the only significant case (N_{ext}, $p = 0.005$) involves absolute differences of ≤ 52 s.

Interval width and its mechanism.. The width $C^{+}_{\max} - C^{-}_{\max}$ measures solution uncertainty [4, 5]. Table 4 shows that LEX2 yields intervals 5.3% narrower

Table 4: Mean makespan interval width by ranking operator (Phase A; mean over 5 neighbourhoods × 82 instances; one best-of-30 solution per configuration).

Operator	Mean width	vs. EV
EV	269.3	—
LEX1	286.4	+6.4%
LEX2	**255.0**	**−5.3%**
YX	267.5	−0.7%

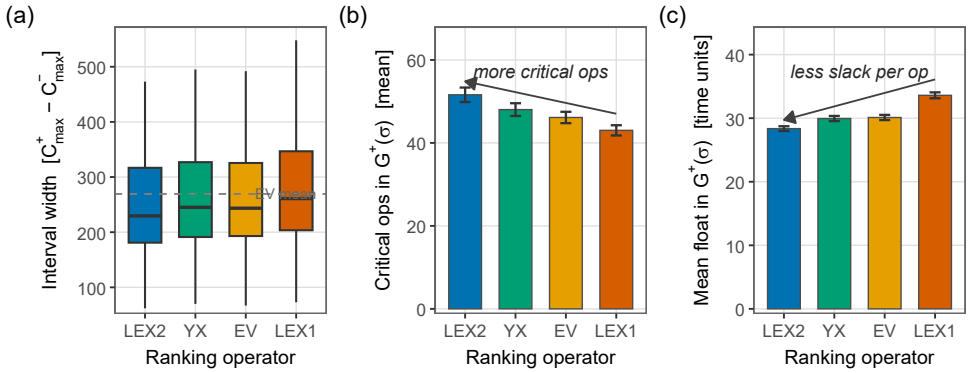


Figure 3: Operator comparison (5 neighbourhoods × 82 instances = 410 observations per operator; one best-of-30 solution per configuration). (a) Distribution of makespan interval width $C^+_{\max} - C^-_{\max}$; the dashed line marks the EV mean. (b) Mean number of critical operations in $G^+(\sigma)$. (c) Mean per-operation float in $G^+(\sigma)$. Panels (b) and (c) refute the slack hypothesis: LEX2 has *more* critical operations and *less* per-operation slack in $G^+(\sigma)$, confirming directional tightening of the worst-case critical path.

than EV and 10.7% narrower than LEX1; the ordering LEX2 $\prec$ YX $\prec$ EV $\prec$ LEX1 holds across every neighbourhood. The effect is consistent at the instance level: LEX2 produces narrower intervals than LEX1 in 88.8%, than EV in 72.2%, and than YX in 70.5% of (instance, neighbourhood) pairs. Figure 3(a) confirms the distributional shift; panels (b) and (c) reveal its mechanism. The natural hypothesis is that LEX2 leaves more structural slack in $G^+(\sigma)$, letting the worst-case path shorten. The float analysis refutes this: LEX2 solutions have *more* critical operations and *less* per-operation float in $G^+(\sigma)$, the opposite of the slack hypothesis. The mechanism is directional: LEX2 accepts moves that reduce $C^+_{\max}$ even when $C^-_{\max}$ stays unchanged, directly tightening the upper-bound graph. This asymmetry between the two extreme graphs is LEX2's structural fingerprint; Section 9.2 provides the theoretical account.

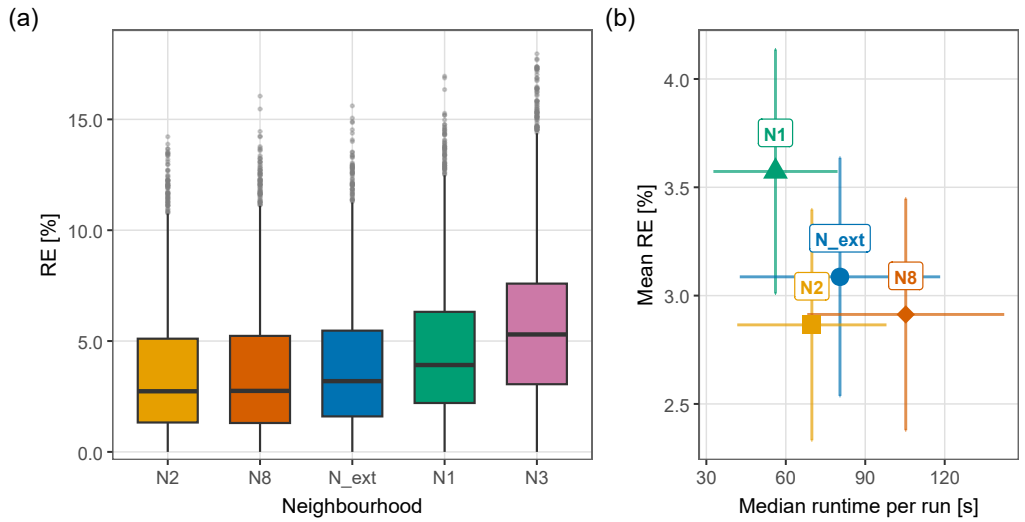


Figure 4: Phase B (irace-tuned tabu search). **(a)** Per-instance RE (%) on the makespan midpoint by neighbourhood (Eq. 12). Each box pools $82 \times 30 = 2{,}460$ observations; pairwise significance and effect sizes are reported in Table 5. **(b)** Quality–runtime trade-off (N3 omitted for scale clarity): each point is the mean RE (%) vs. mean median runtime aggregated over all instance groups; error bars are ± 1 SE. Aggregated over all groups, N_2 achieves the lowest mean RE (%) at moderate runtime ($\approx$70 s); N_8 is slower ($\approx$105 s) with nearly identical RE (%); N_1 is fastest ($\approx$56 s) but incurs a large-effect quality penalty; N_{ext} sits between N_8 and N_1 on both axes.

Conclusion and transition to Phase B.. Although YX attains the best mean midpoint, its edge over LEX2 is small ($r \leq 0.16$) and LEX2 is statistically equivalent to EV ($p = 0.197$); since LEX2 also yields the narrowest intervals (5–11% narrower than the alternatives), all Phase B configurations fix LEX2. This is consistent with prior analyses where, even under per-operator hyper-parameter tuning, LEX2 attained comparable solution quality with greater robustness [24, 32]; the relationship between robustness and interval width (uncertainty) remains to be characterised.

8.3. Phase B: Neighbourhood Comparison after Tuning

Phase B is the central experimental contribution. Each neighbourhood is given its own irace-tuned tabu search configuration (Table 1), isolating structural neighbourhood differences from confounding parameter choices. The switch from hill climbing to tabu search is necessary: hill climbing converges

Table 5: Phase B: pairwise Wilcoxon tests for tuned neighbourhoods ($n = 2460$ paired blocks, Holm–Bonferroni corrected). All differences are statistically significant; for the three N_3 comparisons the p-value underflows double precision, so a representable bound is shown.

A	B	Mean A	Mean B	Diff	p_{adj}	r	Magnitude
N_2	N_3	1846.5	1891.9	-45.4	$< 10^{-300}$	0.844	large
N_3	N_8	1891.9	1847.9	$+44.0$	$< 10^{-300}$	0.835	large
N_3	N_{ext}	1891.9	1853.8	$+38.2$	$< 10^{-300}$	0.807	large
N_1	N_3	1868.6	1891.9	-23.3	$< 10^{-232}$	0.666	large
N_1	N_2	1868.6	1846.5	$+22.1$	$< 10^{-221}$	0.663	large
N_1	N_8	1868.6	1847.9	$+20.7$	$< 10^{-198}$	0.627	large
N_1	N_{ext}	1868.6	1853.8	$+14.9$	$< 10^{-122}$	0.491	medium
N_2	N_{ext}	1846.5	1853.8	-7.3	$< 10^{-61}$	0.356	medium
N_{ext}	N_8	1853.8	1847.9	$+5.8$	$< 10^{-38}$	0.281	small
N_2	N_8	1846.5	1847.9	-1.4	$< 10^{-3}$	0.077	negligible

quickly to a local optimum and then stalls, suppressing any structural difference between neighbourhoods; tabu search escapes local optima throughout the full time budget, allowing each neighbourhood's block geometry and candidate-set size to influence the search trajectory. Using the irace-tuned configurations, I evaluate the five neighbourhoods on the 82 benchmark instances (30 runs per instance, $5 \times 82 \times 30 = 12{,}300$ observations).

Figure 4(a) shows the per-instance relative error RE (%):

$$\text{RE}\,(\%) = \frac{E[\mathbf{C}_{\max}] - \text{LB}}{\text{LB}} \times 100, \tag{12}$$

where LB is the published lower bound of the corresponding *crisp* (deterministic) instance [29, 28, 23]. Since this bound need not coincide with the interval-makespan midpoint, RE (%) is an indicative common yardstick rather than a strict optimality gap; it is used uniformly throughout Sections 8.3–8.4. The Friedman test yields $\chi^2(4) = 3948.37$, $p < 10^{-300}$; mean ranks are N_2: 2.132, N_8: 2.240, N_{ext}: 2.650, N_1: 3.441, N_3: 4.538. Pairwise tests (Table 5) reveal that N_2 and N_8 are essentially tied (negligible effect, $r = 0.08$, $p < 10^{-3}$), while both are separated from N_{ext} by a medium effect, from N_1 by a large effect, and from N_3 by a very large margin.

Figure 4(b) visualises the quality–runtime trade-off aggregated across instance groups: N_2 achieves the lowest mean RE (%) at moderate runtime, with N_8 nearly indistinguishable in quality at slightly higher cost; N_1 is the

Table 6: Phase B: mean RE (%) and median runtime t (s) per instance group and neighbourhood (irace-tuned TS, 30 runs per instance). Bold: minimum RE (%) in each row. The Classical row aggregates the 12 OR-Library instances; Taillard rows aggregate the 10 instances of each size class. Per-instance results are provided as supplementary material [31].

		N_2		N_8		N_{ext}		N_1		N_3	
Group	n	RE	t	RE	t	RE	t	RE	t	RE	t
Classical	12	**2.40**	18	2.43	30	2.52	14	2.86	12	3.91	77
15×15	10	2.04	19	**1.95**	33	2.24	14	2.38	11	3.14	76
20×15	10	**3.58**	42	3.69	65	3.85	32	4.28	24	5.40	175
20×20	10	**5.34**	58	5.42	96	5.69	44	5.86	33	6.71	274
30×15	10	**3.68**	103	3.80	172	4.11	96	5.00	72	6.27	464
30×20	10	**8.62**	161	8.85	266	9.07	122	9.95	100	11.87	853
50×15	10	**0.32**	173	0.33	260	0.61	345	1.48	279	2.58	897
50×20	10	**2.03**	535	2.13	695	2.82	680	4.63	388	6.45	898
Grand mean		**3.47**	60	3.55	100	3.83	46	4.51	37	5.75	297

fastest but sits at substantially worse quality (a large effect separates it from N_2).

Decomposing by instance group (Table 6), N_2 posts the lowest group-level RE (%) on 7 of 8 groups—the aggregated Classical row and all Taillard size classes from tai20×15 to tai50×20 inclusive—with N_8 leading only on the smallest tai15×15 class. N_{ext}, N_1 and N_3 never win at the group level; per-instance results (where N_2 wins 49 of the 82 individual instances, N_8 wins 25, and N_{ext} wins 8 — including LA24 and LA38) are provided in the supplementary material [31]. On runtime, the picture varies by instance size: N_1 is the fastest neighbourhood on the small and medium Taillard classes (tai30×20: 100 s vs N_2's 161 s) but trails on quality by a large effect; Among the four neighbourhoods that finish without hitting the time limit, N_{ext} becomes the slowest on the largest instances (tai50×15: 345 s; tai50×20: 680 s), exceeding N_2's runtimes (173 s; 535 s) by a factor of ≈2 at inferior quality. N_3 again occupies the worst position on both dimensions: its median runtime reaches 898 s at tai50×20—effectively hitting the 900 s time limit—with the worst quality on every instance group.

Considered jointly, N_2 offers the best efficiency profile across all instance sizes: it ties with N_8 for the best quality and is the faster of the two on every group, while being faster than N_{ext} on instances with ≥ 750 operations. N_8 is a competitive alternative on the smallest tai15×15 class, where it achieves

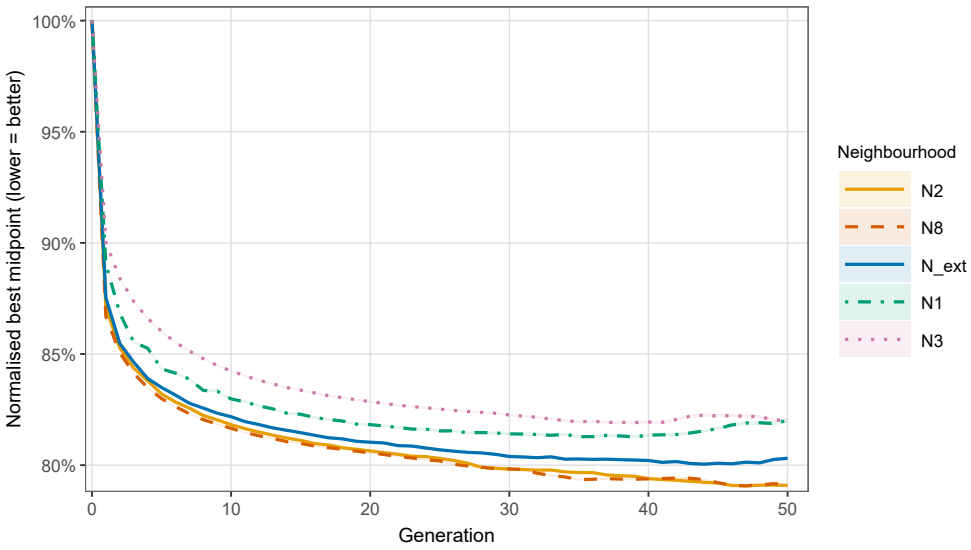


Figure 5: Per-generation normalised best-midpoint profiles, by neighbourhood (Phase B, irace-tuned tabu search; mean over 82 instances × 30 runs per neighbourhood; each run is normalised by its generation-0 value). The shaded ribbons represent ±1 standard error.

the lowest RE (%) (1.95% vs N_2's 2.04%); on every other class N_2 wins by a small to moderate margin. The practical implication is that N_2 is the default choice for most settings; N_8 may be a viable alternative on small instances where its extra-block moves can offset its higher per-iteration cost.

8.3.1. Convergence Profiles

Figure 5 shows the per-generation convergence profiles of the five neighbourhoods, each under its irace-tuned configuration, averaged over all 82 instances and 30 runs. Each run's best midpoint is divided by its generation-0 value, so every curve starts at 1.0 and decreases. The bulk of the improvement is captured very early: all curves cover ∼83–86% of their total reduction by generation 5 and ∼90–93% by generation 10, and the neighbourhood ranking is already set from the first few generations. By generation 25 the curves have largely stabilised: N_2 and N_8 are essentially tied at the lowest values (0.803 and 0.802 respectively), N_{ext} sits at 0.807, N_1 at 0.815, and N_3 at 0.825, consistent with the statistical lead of N_2 and N_8 in Table 5.

Table 7: Comparison with published IJSP solvers on 12 classical instances (Phase B, irace-tuned TS, N_2). Metric: RE (%) $= (E[\mathbf{C}_{\max}] - \mathrm{LB}) / \mathrm{LB} \times 100$ (Eq. 12). LBs from [23]. GA from [18]; *fEABC* from [23]; *ESABC* from [24]. All methods: 30 independent runs. Bold: row-minimum average RE. t = median runtime (s); the TS-N_2 runtimes were measured on a workstation approximately 2× slower than the hardware used in the cited published solvers (see Section 8.2 for the calibration), so direct t comparisons should be interpreted accordingly. RE (%) is hardware-independent.

		GA [18]			*fEABC* [23]			*ESABC* [24]			*TS-N_2* (ours)		
Instance	LB	Best	Avg	t	Best	Avg	t	Best	Avg	t	Best	Avg	t
ABZ7	656	6.33	12.50	1.8	5.26	7.32	4.5	4.19	6.73	6.5	1.83	**3.03**	32
ABZ8	645	11.32	18.48	1.8	8.99	12.06	4.2	8.68	10.95	7.7	4.50	**6.64**	39
ABZ9	661	13.01	17.97	2.2	9.68	13.13	6.0	8.55	11.19	8.7	5.07	**6.31**	40
FT10	930	1.83	5.23	0.5	1.08	4.11	1.6	0.59	3.01	1.8	0.59	**0.99**	7
FT20	1165	1.46	4.35	0.7	0.69	1.73	2.7	0.69	1.78	2.9	0.09	**0.23**	9
la21	1046	3.15	5.01	1.1	2.58	5.01	1.8	2.06	3.96	2.9	0.57	**1.39**	11
la24	935	4.06	6.34	0.8	2.25	5.06	2.7	3.53	4.95	2.6	0.80	**1.37**	10
la25	977	1.94	5.11	1.0	1.94	3.88	2.4	1.33	2.74	3.0	0.20	**1.31**	10
la27	1235	4.57	10.22	1.3	2.75	4.66	4.1	2.75	4.12	5.8	0.97	**1.69**	19
la29	1152	11.11	14.23	1.1	5.51	8.65	4.4	4.99	7.03	6.7	1.95	**2.87**	21
la38	1196	6.02	9.16	1.4	4.52	6.88	6.0	3.01	5.83	7.2	0.71	**1.91**	20
la40	1222	5.07	8.74	1.2	1.88	4.21	3.0	2.74	4.11	3.7	0.70	**1.09**	17
Mean			9.78			6.39			5.53			**2.40**	

8.4. Comparison with Published IJSP Solvers

I compare the irace-tuned TS under neighbourhood N_2 (Phase B best, Section 8.3) against three published IJSP metaheuristics on the same benchmark instances: a Genetic Algorithm (GA) [18], a fast-elitist Artificial Bee Colony (*fEABC*) [23], and an Elitist-Selection ABC (*ESABC*) [24]. All methods use 30 independent runs per instance; results for the published solvers are taken directly from the cited papers. The comparison metric is RE (%) (Equation 12), where the LB values are from [23] for the classical instances and from [29] for the Taillard instances.

Classical instances.. Table 7 compares all four solvers on the 12 classical instances (ABZ7–9, FT10, FT20, LA21/24/25/27/29/38/40). TS-N_2 achieves a mean RE of **2.40**%, against 5.53% for ESABC, 6.39% for *fEABC*, and 9.78% for GA, a reduction of 56.6% relative to the previous best (ESABC). TS-N_2 attains the lowest mean RE on all 12 instances individually; the largest relative gains arise on FT20 (0.23% vs. 1.78% for ESABC, an 87% reduction)

and la40 (1.09% vs. 4.11%, a 73% reduction).

Taillard instances.. Per-instance results on all 70 Taillard instances (TA1–TA70, seven size classes from 15×15 to 50×20) are provided as supplementary material [31]; Figure 6 summarises mean RE (%) per size class. GA results are unavailable for the Taillard IJSP instances; the comparison covers *fEABC* and ESABC only; TS-N_8 is included alongside TS-N_2 for reference. TS-N_2 achieves a grand mean RE of **3.66**% across all 70 instances, against 8.66% for ESABC and 9.39% for *fEABC*, a 57.7% reduction relative to ESABC. TS-N_2 achieves the strictly lower average RE than both ABC methods on every one of the 70 instances, and the improvement is consistent across size classes (Figure 6; per-instance values in [31]). TS-N_8 finishes marginally behind TS-N_2 in grand mean (3.74% vs. 3.66%), consistent with the Phase B ranking (Section 8.3).

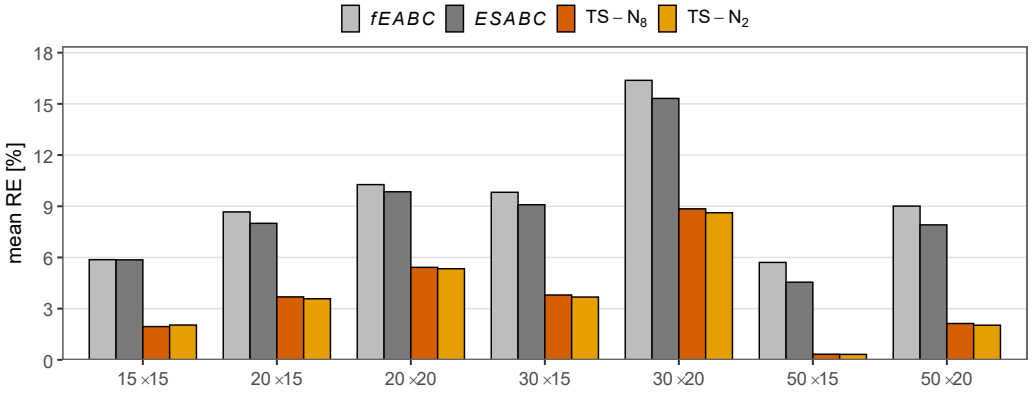


Figure 6: Mean RE (%) per Taillard size class for the four solvers compared on the 70 Taillard IJSP instances (30 runs per instance). The irace-tuned TS variants reduce mean RE relative to ESABC across all classes (range 44%–93%; 57.7% on the grand mean). Per-instance values are provided as supplementary material [31].

9. Discussion

9.1. Practical Recommendations

The experimental results lead to a small set of actionable recommendations:

1. Choose between N_2 and N_8 based on instance size and time budget. N_2 is the default recommendation: it ties with N_8 for the best quality overall (negligible difference, $r = 0.08$) and is faster on every group of the benchmark (Table 6). Only on the smallest tai15×15 class, N_8 achieves the strictly lower group mean RE (%) (1.95% vs N_2's 2.04%) at modest extra runtime (33 s vs 19 s).
2. Use LEX2 if interval width is the goal; operator choice has limited impact on midpoint. LEX2 yields makespan intervals 5–11% narrower than the alternatives on average, with structural evidence for this effect (Section 9.2), and this advantage is consistent across prior work that includes per-operator hyperparameter tuning [24, 32].
3. Avoid N_3 in either setting. The 3-task block rearrangements it adds beyond N_2 are dominated on both dimensions: it is the weakest neighbourhood on solution quality and the most expensive computationally, reaching the 900 s time limit for the largest instances.

9.2. Theoretical Connection: Why LEX2 Narrows Intervals

Proposition 2 guarantees that reversing a non-extreme-critical arc cannot improve either bound. LEX2 specifically targets $C^+_{\max}$, accepting moves that reduce $C^+_{\max}$ even when $C^-_{\max}$ stays put. Over many local search steps, this creates a bias toward configurations where $G^+(\sigma)$ has a shorter critical path — and, by the structure of interval arithmetic, a tighter bound on the overall interval width. The float analysis of Section 8.2 provides direct empirical confirmation: LEX2 solutions have more critical operations in $G^+(\sigma)$ and less per-operation slack. This concentration of criticality in $G^+(\sigma)$—the graph LEX2 directly targets—is a structural fingerprint of the ranking rule.

9.3. Why N_3 Underperforms

N_3 is counter-intuitive: it is a strict superset of N_2 in the move-generation sense (boundary arc swaps plus three types of 3-task block rearrangements—cyclic left-shift, cyclic right-shift, and full reversal of boundary triples), yet it produces worse solutions. Each 3-task rearrangement candidate requires a full feasibility evaluation to detect cycles before it can be retained, which significantly increases the per-iteration cost. More importantly, these extra candidates do not carry improving moves in practice: the block-boundary arc swaps inherited from N_2 already capture the structurally meaningful transitions in the interval-extended Nowicki–Smutnicki framework (Section 6.2), and the 3-task rearrangements add volume to the candidate pool without

adding quality. The penalty manifests in both phases: under first-improvement hill climbing the larger pool inflates the per-iteration cost (Table 2: 254 s vs 33 s for N_2 at LEX2) at a modest quality loss; under best-improvement tabu search the dilution is sharper, as the rearrangements compete for selection at each iteration, displacing higher-yield boundary moves and producing both slower convergence and a worse plateau (Table 6).

9.4. Why N_8 Performs Well Under Tabu Search

N_8 extends N_2 with extra-block reinsertion moves filtered by a heads-and-tails interval estimate. Under first-improvement hill climbing (Phase A), the reinsertion moves add evaluation cost without sufficient benefit: the search reaches a local optimum quickly, where the larger candidate set yields few new improvements. The result is slightly worse quality and longer runtime than N_2 (Table 2).

Under tabu search (Phase B) this penalty disappears. Three interacting mechanisms plausibly contribute: the tabu list prevents cycling near a local optimum, forcing the search to explore the full breadth of N_8's candidate set, including the reinsertion moves; best-improvement selection in TS captures improvements in this larger pool that first-improvement would miss; and the aspiration criterion admits otherwise-tabu reinsertion moves that improve the interval criterion in aggregate. The search-strategy dependence observed here may be specific to interval uncertainty, where G^- and G^+ can disagree on which moves are most valuable, in contrast to the deterministic setting [11] where a single critical path determines makespan.

9.5. Limitations and Future Work

Several directions remain open. The benchmark could be extended to tai100×20 (2000 operations) and other 100-job classes, where larger search spaces tend to sharpen structural differences between neighbourhoods; the question of whether ranking-operator differences persist under per-operator tuning also remains open beyond the 12 small instances studied in [24, 32]. The ±7.5% uniform interval used here is a single noise level, and characterising the relationship between input uncertainty and solution robustness remains open; in particular, the present data together with the LEX2 robustness advantage reported in [24, 32] suggest that the output interval width $C^+_{\max} - C^-_{\max}$ could itself serve as an a priori robustness indicator, which would be worth confirming across a broader range of interval widths. Finally, $(\text{LEX2}, N_2)$ is the natural starting point for a dedicated uncertainty-aware

metaheuristic: LEX2 directly tightens the worst-case bound (Section 9.2) and N_2 provides the strongest neighbourhood structure overall. A dedicated algorithm that exploits the structural mechanism of Section 9.2 could improve on the general-purpose tabu search used here.

10. Conclusions

I have presented a theoretical framework for neighbourhood-based local search in the Interval Job Shop Scheduling Problem grounded in the concept of *extreme critical paths* — arcs that are critical in either the best-case or worst-case scenario graph. The resulting neighbourhood $H(\sigma)$ satisfies feasibility, connectivity and the no-loss-of-improving-neighbours property for all four admissible interval rankings considered (EV, LEX1, LEX2, YX) simultaneously.

Five neighbourhood variants (N_1, N_2, N_3, N_{ext}, N_8) and four ranking operators were evaluated on 82 benchmark instances of up to 1000 operations, across three phases: a 5×4 comparison at a common parameter setting, a five-way comparison after irace-based tuning, and a comparison with three published IJSP metaheuristics.

After per-neighbourhood irace-tuned tabu search, N_2 and N_8 emerge as the two strongest variants—statistically tied (negligible effect $r = 0.08$)—while N_{ext} trails by a medium effect ($r = 0.36$) and N_1 and N_3 by large effects ($r = 0.66$ and 0.84; Table 5). Considered jointly with runtime, N_2 is the most efficient default: it ties with N_8 for best quality but is uniformly faster, and is faster than N_{ext} on instances with ≥ 750 operations. Only on the tai15×15 class does N_8 achieve the strictly lower mean RE, at modest extra runtime (33 s vs N_2's 19 s). Regarding ranking operators, LEX2 yields makespan intervals 5–11% narrower than the alternatives on average. A float analysis of both extreme graphs provides structural support for this finding: LEX2 solutions show more critical operations and less per-operation slack in $G^+(\sigma)$, consistent with LEX2 directly tightening the worst-case critical path rather than leaving more structural slack. The comparison against three published IJSP metaheuristics (GA, *fEABC*, ESABC) provides external validation of the approach: the irace-tuned TS-N_2 achieves strictly lower average RE (%) than the previous best solver on every one of the 82 benchmark instances, with mean reductions of 56.6% on the 12 classical instances and 57.7% on the 70 Taillard instances.

These results provide actionable guidance for practitioners implementing local search for uncertain job shop scheduling: the choice of ranking operator can be used not only to optimise expected makespan but also to directly control solution uncertainty, and the choice of neighbourhood should account for the local-search strategy used.

CRediT authorship contribution statement

Hernán Díaz Rodríguez: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft, Writing – review and editing.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The raw experimental data underlying every figure and table in this article (run logs, aggregated CSVs, irace tuning records, instance files, and analysis scripts) are openly available on Zenodo [33]. Per-instance RE (%) results for all 70 Taillard instances are provided as a separate supplementary document [31]. The classical JSP benchmarks used to derive the interval instances are taken from the OR-Library and from [34].

Acknowledgements

This work was supported by the Spanish Ministry of Science, Innovation and Universities (MCIN/AEI/10.13039/501100011033) under grant PID2022-141746OB-I00.

References

[1] P. J. M. van Laarhoven, E. H. L. Aarts, J. K. Lenstra, Job shop scheduling by simulated annealing, Operations Research 40 (1) (1992) 113–125. doi:10.1287/opre.40.1.113.

[2] E. Nowicki, C. Smutnicki, A fast taboo search algorithm for the job shop problem, Management Science 42 (6) (1996) 797–813. doi:10.1287/mnsc.42.6.797.

[3] E. Nowicki, C. Smutnicki, An advanced tabu search algorithm for the job shop problem, Journal of Scheduling 8 (2005) 145–159. doi:10.1007/s10951-005-6364-5.

[4] R. E. Moore, R. B. Kearfott, M. J. Cloud, Introduction to Interval Analysis, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2009. doi:10.1137/1.9780898717716.

[5] D. Lei, Interval job shop scheduling problems, International Journal of Advanced Manufacturing Technology 60 (2012) 291–301. doi:10.1007/s00170-011-3600-3.

[6] H. Bustince, J. Fernandez, A. Kolesárová, R. Mesiar, Generation of linear orders for intervals by means of aggregation functions, Fuzzy Sets and Systems 220 (2013) 69–77. doi:10.1016/j.fss.2012.07.015.

[7] S. Karmakar, A. K. Bhunia, A comparative study of different order relations of intervals, Reliable Computing 16 (2012) 38–72.

[8] D. Dubois, H. Fargier, P. Fortemps, Fuzzy scheduling: Modelling flexible constraints vs. coping with incomplete knowledge, European Journal of Operational Research 147 (2) (2003) 231–252. doi:10.1016/S0377-2217(02)00558-1.

[9] C. Artigues, C. Briand, T. Garaix, Temporal analysis of projects under interval uncertainty, in: C. Schwindt, J. Zimmermann (Eds.), Handbook on Project Management and Scheduling Vol. 2, Springer, 2015, pp. 911–927. doi:10.1007/978-3-319-05915-0_11.

[10] I. González Rodríguez, C. R. Vela, J. Puente, R. Varela, A new local search for the job shop problem with uncertain durations, in: Proceedings of the Eighteenth International Conference on Automated Planning and Scheduling (ICAPS-2008), AAAI Press, Sydney, Australia, 2008, pp. 124–131.

[11] J. Xie, X. Li, L. Gao, L. Gui, A new neighbourhood structure for job shop scheduling problems, International Journal of Production Research 61 (7) (2022) 2147–2161. doi:10.1080/00207543.2022.2060772.

[12] Z. Xu, R. R. Yager, Some geometric aggregation operators based on intuitionistic fuzzy sets, International Journal of General Systems 35 (4) (2006) 417–433. doi:10.1080/03081070600574353.

[13] I. González Rodríguez, C. R. Vela, J. Puente, A. Hernández-Arauzo, Improved local search for job shop scheduling with uncertain durations, Proceedings of the International Conference on Automated Planning and Scheduling 19 (1) (2009) 154–161. doi:10.1609/icaps.v19i1.13371.

[14] J. Puente, C. R. Vela, I. González Rodríguez, Fast local search for fuzzy job shop scheduling, in: H. Coelho, et al. (Eds.), Proceedings of the 19th European Conference on Artificial Intelligence (ECAI 2010), IOS Press, 2010, pp. 739–744. doi:10.3233/978-1-60750-606-5-739.

[15] C. R. Vela, S. Afsar, J. J. Palacios, I. González-Rodríguez, J. Puente, Evolutionary tabu search for flexible due-date satisfaction in fuzzy job shop scheduling, Computers & Operations Research 119 (2020) 104931. doi:10.1016/j.cor.2020.104931.

[16] P. García Gómez, J. J. Palacios, C. R. Vela, I. González-Rodríguez, Adaptive lexicographical optimisation with vague goals in green fuzzy flexible job shop scheduling, Annals of Operations Research (2026). doi:10.1007/s10479-026-07239-1.

[17] S. Afşar, J. Puente, J. J. Palacios, I. González-Rodríguez, C. R. Vela, A hybrid evolutionary approach for lexicographic green flexible jobshop with interval uncertainty, Natural Computing 24 (3) (2025) 483–496. doi:10.1007/s11047-025-10016-x.

[18] H. Díaz, I. González-Rodríguez, J. J. Palacios, I. Díaz, C. R. Vela, A genetic approach to the job shop scheduling problem with interval uncertainty, in: Information Processing and Management of Uncertainty in Knowledge-Based Systems (IPMU 2020), Vol. 1238 of Communications in Computer and Information Science, Springer, 2020, pp. 663–676. doi:10.1007/978-3-030-50143-3_52.

[19] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Tardiness minimisation for job shop scheduling with interval uncertainty, in: Hybrid Artificial Intelligence Systems (HAIS 2020), Vol. 12344 of Lecture Notes in Artificial Intelligence, Springer, 2020, pp. 209–220. doi:10.1007/978-3-030-61705-9_18.

[20] K. Tamssaouet, S. Dauzère-Pérès, A general efficient neighborhood structure framework for the job-shop and flexible job-shop scheduling problems, European Journal of Operational Research 311 (2) (2023) 455–471. doi:10.1016/j.ejor.2023.05.018.

[21] P. García Gómez, S. Dauzère-Pérès, C. R. Vela, I. González-Rodríguez, New bounds for move evaluation in the flexible job-shop scheduling problem, Computers & Industrial Engineering 212 (2026) 111696. doi:10.1016/j.cie.2025.111696.

[22] A. Calamita, C. Meloni, M. Pranzo, M. Samà, Fast risk estimation for job shop scheduling solutions under interval uncertainty via machine learning, Flexible Services and Manufacturing Journal (2025). doi:10.1007/s10696-025-09640-7.

[23] H. Diaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, Fast elitist ABC for makespan optimisation in interval JSP, Natural Computing 22 (4) (2023) 645–657. doi:10.1007/s11047-023-09953-2.

[24] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela, An elitist seasonal artificial bee colony algorithm for the interval job shop, Integrated Computer-Aided Engineering 30 (3) (2023) 223–242. doi:10.3233/ICA-230705.

[25] J. Fortin, P. Zieliński, D. Dubois, H. Fargier, Criticality analysis of activity networks under interval uncertainty, Journal of Scheduling 13 (6) (2010) 609–627. doi:10.1007/s10951-010-0163-3.

[26] D. Dubois, H. Fargier, J. Fortin, Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks, Journal of Intelligent Manufacturing 16 (4–5) (2005) 407–421. doi:10.1007/s10845-005-1654-5.

[27] T. Garaix, C. Artigues, C. Briand, Fast minimum float computation in activity networks under interval uncertainty, Journal of Scheduling 16 (1) (2013) 93–103. doi:10.1007/s10951-012-0272-2.

[28] J. E. Beasley, OR-Library: distributing test problems by electronic mail, Journal of the Operational Research Society 41 (11) (1990) 1069–1072. doi:10.1057/jors.1990.166.

[29] É. Taillard, Benchmarks for basic scheduling problems, European Journal of Operational Research 64 (2) (1993) 278–285. doi:10.1016/0377-2217(93)90182-M.

[30] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, M. Birattari, T. Stützle, The irace package: Iterated racing for automatic algorithm configuration, Operations Research Perspectives 3 (2016) 43–58. doi:10.1016/j.orp.2016.09.002.

[31] H. Díaz Rodríguez, Supplementary material: per-instance RE (%) on Taillard IJSP instances (TA1–TA70) (2026). doi:10.5281/zenodo.20511981.
URL https://doi.org/10.5281/zenodo.20511981

[32] H. Díaz, J. J. Palacios, I. Díaz, C. R. Vela, I. González-Rodríguez, Robust schedules for tardiness optimisation in job shop with interval uncertainty, Logic Journal of the IGPL 31 (2) (2022) 240–254. doi:10.1093/jigpal/jzac016.

[33] H. Díaz Rodríguez, Experimental dataset: Neighbourhood structures and ranking operators for the interval JSP (raw runs, configurations, instances, scripts) (2026). doi:10.5281/zenodo.20511562.
URL https://doi.org/10.5281/zenodo.20511562

[34] Optimizizer, Best known solutions for the Taillard job shop instances, http://optimizizer.com/TA.php, accessed: May 2026.